3.8. Безопасное программирование защита от ошибок и атак

3. Сети, базы данных, безопасность кода

В этом уроке

- Разберемся с принципами DevSecOps
- Узнаем, что такое проверка ввода и валидация
- Разработаем безопасную форму входа

Зачем нужно безопасное программирование

- Устранение уязвимостей до эксплуатации
- Снижение рисков утечки данных
- Соответствие стандартам безопасности
- Защита пользователей и инфраструктуры
- Предотвращение уязвимостей и атак
- Повышение надежности приложений
- Защита персональных данных пользователей
- Соответствие нормативным требованиям (GDPR, 152-ФЗ)

Проверка и валидация ввода

- Никогда не доверяй пользовательскому вводу
- Проверка типов, длины, формата
- Белые списки (разрешенные значения) предпочтительнее черных
  - Пример: email, номер телефона, пароль
- Очистка ввода: удаление специальных символов
- Запрет ввода HTML и SQL-кода

Пример валидации на Python

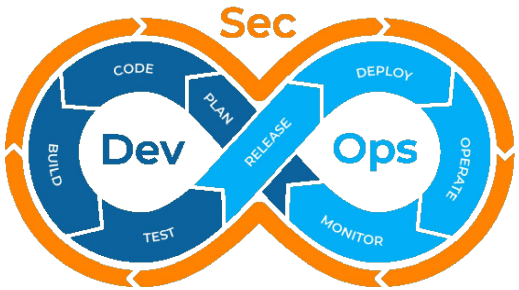
```
import re


def is_valid_email(email):
    return re.match(r"^[\w.-]+@[\w.-]+\.\w+$", email) is not None
```

Безопасная разработка кода

Приказ ФСТЭК России №239 от 25.12.17 предъявляет Требования по безопасной разработке программного обеспечения (29.3.1.)
ГОСТ Р 56939-2016 «Разработка безопасного ПО. Общие требования»
В июне 2023 г. МинЦифры и ФСТЭК России предложили создать единую среду безопасной разработки

.

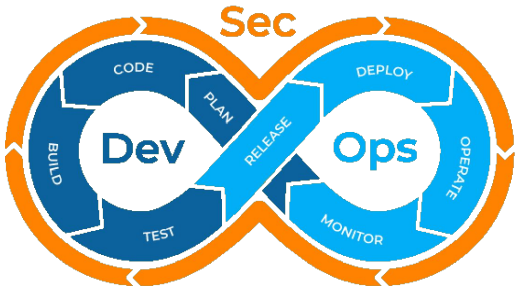


https://docs.cntd.ru/document/1200135525

Безопасная разработка кода

DevSecOps – это практика интеграции тестирования безопасности в каждый этап процесса разработки программного обеспечения.
Обязанности:

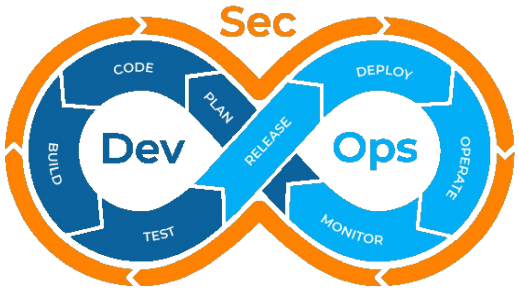
- Анализ кода
- Управление изменениями
- Управление соответствием
- Моделирование угроз
- Обучение безопасности



https://docs.cntd.ru/document/1200135525

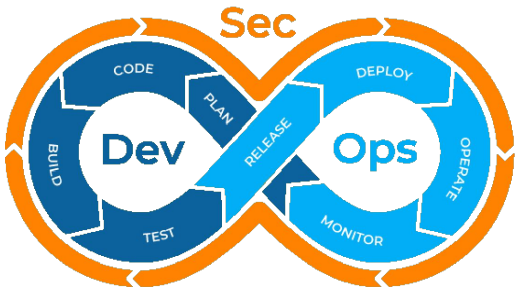
Принципы безопасного кода (DevSecOps)

- Безопасность — часть процесса разработки
- Автоматическое тестирование на уязвимости
- Минимизация прав доступа
- Использование проверенных библиотек
- Логирование и мониторинг



Принципы DevSecOps

- Security by Design
- Автоматическая проверка кода
- Статический анализ (SAST)
- Интеграция безопасности на каждом этапе разработки



Структура DevSecOps

Виды проверок:

Pre-commit Checks. Проверка кода на наличие конфиденциальной информации (пароли, секреты, токены и т. д.), которая не должна попасть в историю Git.

Pre build Checks. Проверки, которые запускаются при выполнении коммита для контроля корректности и безопасности кода в репозитории.

Post-build Checks. Проверки, которые выполняются после сборки приложения и включают в себя тестирование артефактов сборки (например, docker-образов).

Test-time Checks. Этап тестирования развернутого приложения на наличие уязвимостей (например, для сканирования API на предмет популярных уязвимостей).

Deploy-time Checks. Выполняются при развертывании приложения и проверяют инфраструктуру на предмет уязвимостей.

Pre-commit

Pre-commit — проверка безопасности исходного кода до того, как разработчик закоммитит код в систему контроля версий. Проверка позволяет обнаружить незашифрованную конфиденциальную информацию: пароли или API-ключи.

Инструменты:

- Gitleaks
- git-secrets
- Trivy Secret Scanning
- Scan
- Whispers
- git-all-secrets
- detect-secrets
- gittyleaks

Git Hooks

Pre-build

Pre-build — проверка безопасности исходного кода после того, как он попал в систему контроля версий. Инструменты выполняют статический анализ исходного кода и его зависимостей, чтобы обнаружить распространенные уязвимости, например, многие из списка OWASP Top Ten.

Типы проверок:

- Secret Detection;
- Static Application Security Testing (SAST);
- Software Composition Analysis (SCA).

Инструменты:

- Gitleaks
- **Yandex lockbox**
- Hashicorp Vault
- GitLab SAST
- SonarQube
- **PT Application Inspector**
- **PVS Studio**
- Trivy
- FOSSA
- **Profiscope CodeScoring**

Post-build

Post-build — проверка безопасности артефакта, собранного из исходного кода, например, Docker-файла, RPM-пакета или JAR-архива. Инструменты анализируют окружение и зависимости приложения, отыскивая устаревшие версии пакетов и библиотек, уже имеющих исправления в более новых версиях.

Инструменты:

- GitLab Container Scanning
- Trivy
- Clair
- Grype
- Quay
- JFrog Xray
- Docker Scout

Test-time

Test-time— проверка безопасности приложения, которое запущено из собранного артефакта. Инструменты в этой фазе пытаются «сломать» приложение, имитируя действия злоумышленников.

Типы проверок:

- DAST
- IAST
- OAST

Инструменты:

- GitLab DAST
- OWASP Zed Attack Proxy
- HCL Security AppScan
- Synopsys Managed DAST
- Synopsys Seeker
- Tenable. io (Web App Scanning)
- Veracode Dynamic Analysis
- Veracode Interactive Analysis
- Contrast Assess
- Hdiv Detection (IAST)
- PT Application Inspector
- Burp Suite
- OWASP ZAP

Post-deploy

Post-deploy — проверка безопасности приложения после развёртывания в production-среде. Инструменты в режиме реального времени мониторят открытые реестры известных уязвимостей и отправляют уведомление, обнаружив угрозу для приложения.

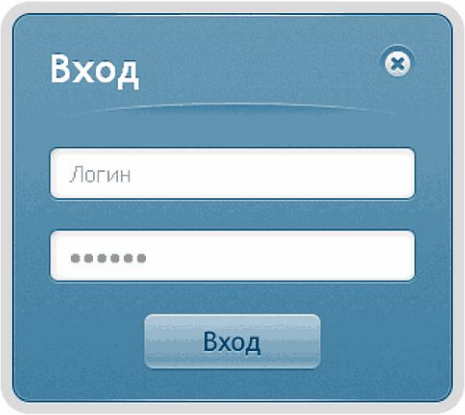
Инструменты:

- Snyk Container
- Trivy
- GitLab Container Scanning

Git Hooks

Безопасная форма входа: принципы

- Использование POST-запросов, а не GET
- Ограничение количества попыток входа (защита от перебора)
- Валидация логина и пароля до отправки
- Защита от XSS и CSRF-атак
- Хранение паролей с хэшированием (bcrypt, argon2)



Пример безопасной формы

```
if not login.isalnum():
    raise ValueError('Недопустимые символы')

if not strong_password(password):
    raise ValueError('Слабый пароль')

# Параметризованный SQL-запрос
cursor.execute('SELECT * FROM users WHERE login=? AND password=?', (login, password))
```

Заключение

- Безопасность начинается с кода
- Проверка ввода — первый барьер защиты
- DevSecOps помогает встроить защиту в процесс разработки
- Используйте безопасные практики при работе с формами и базами данных

Итоги урока

- Разобрались с принципами безопасной разработки
- Узнали, что такое проверка ввода и валидация
- Разработали безопасную форму входа